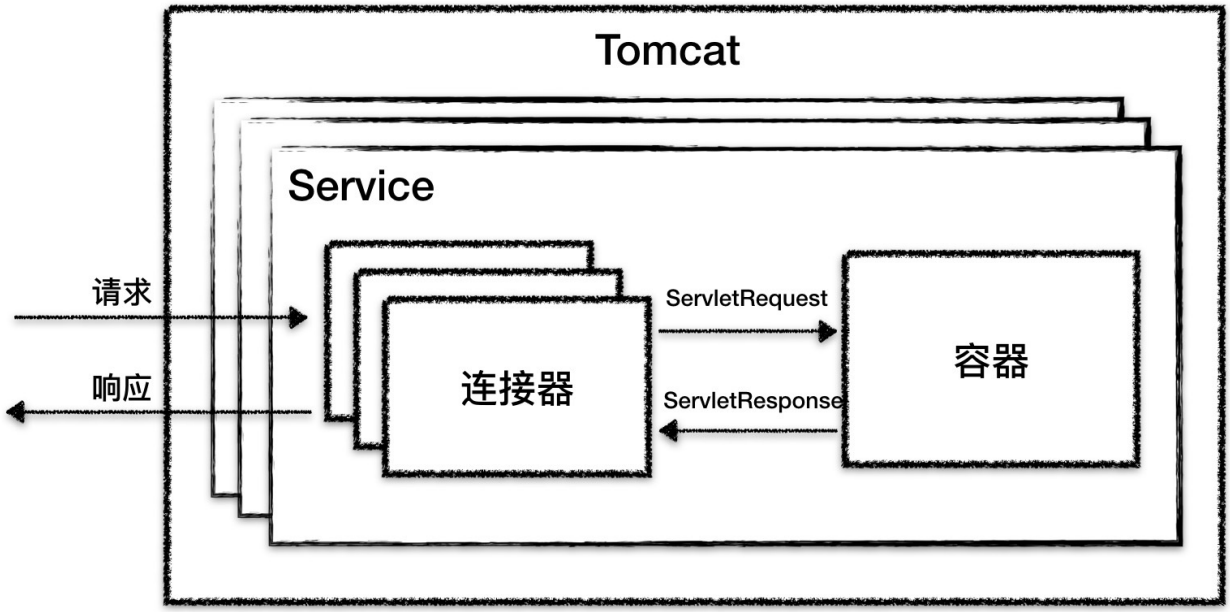
06-Tomcat系统架构（下）：聊聊多层容器的设计

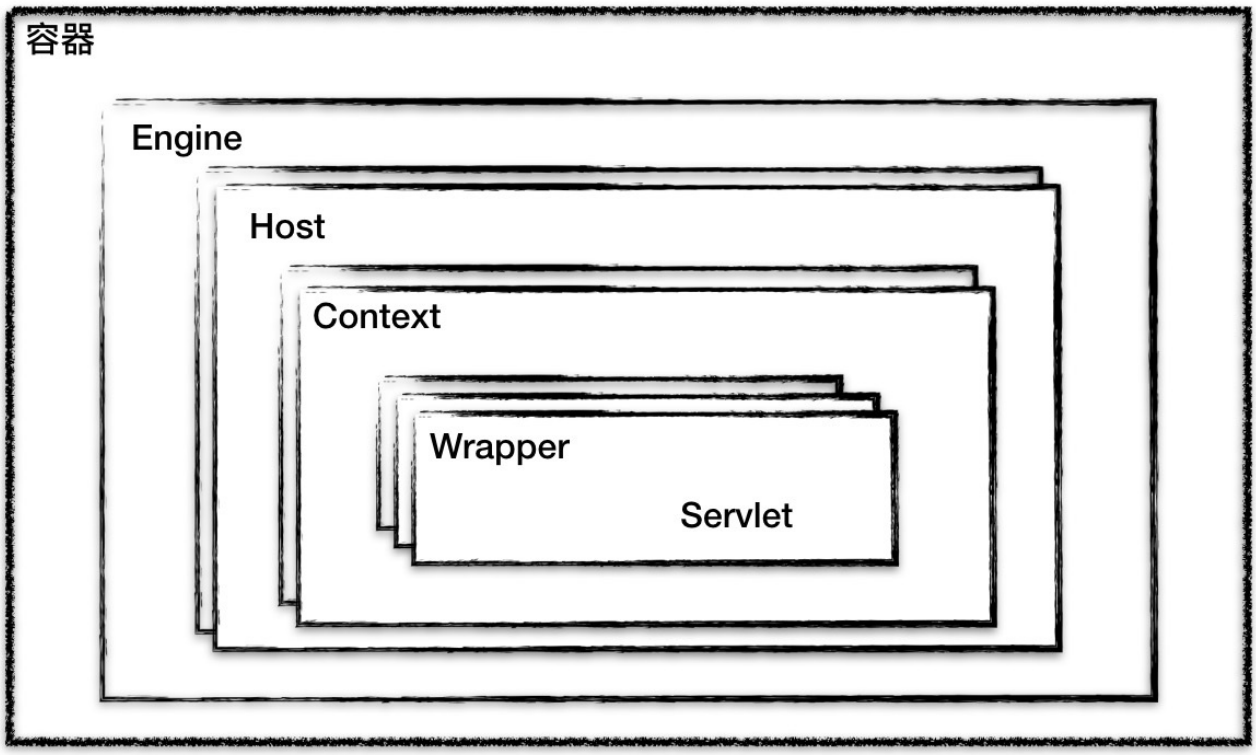
专栏上一期我们学完了连接器的设计，今天我们一起来看一下Tomcat的容器设计。先复习一下，上期我讲到了Tomcat有两个核心组件：连接器和容器，其中连接器负责外部交流，容器负责内部处理。具体来说就是，连接器处理Socket通信和应用层协议的解析，得到Servlet请求；而容器则负责处理Servlet请求。我们通过下面这张图来回忆一下。



容器，顾名思义就是用来装载东西的器具，在Tomcat里，容器就是用来装载Servlet的。那Tomcat的Servlet容器是如何设计的呢？

容器的层次结构

Tomcat设计了4种容器，分别是Engine、Host、Context和Wrapper。这4种容器不是平行关系，而是父子关系。下面我画了一张图帮你理解它们的关系。



仅供个人学习 请勿传播 用户标记 mannix小童鞋

你可能会问，为什么要设计成这么多层次的容器，这不是增加了复杂度吗？其实这背后的考虑是，**Tomcat通过一种分层的架构，使得Servlet容器具有很好的灵活性。**

Context表示一个Web应用程序；Wrapper表示一个Servlet，一个Web应用程序中可能会有多个Servlet；Host代表的是一个虚拟主机，或者说一个站点，可以给Tomcat配置多个虚拟主机地址，而一个虚拟主机下可以部署多个Web应用程序；Engine表示引擎，用来管理多个虚拟站点，一个Service最多只能有一个Engine。

你可以再通过Tomcat的server.xml配置文件来加深对Tomcat容器的理解。Tomcat采用了组件化的设计，它的构成组件都是可配置的，其中最外层的是Server，其他组件按照一定的格式要求配置在这个顶层容器中。

```
<Server>                    //顶层组件，可以包括多个Service
  <Service>                 //顶层组件，可包含一个Engine，多个连接器
    <Connector>             //连接器组件，代表通信接口
    </Connector>
    <Engine>                //容器组件，一个Engine组件处理Service中的所有请求，包含多个Host
      <Host>                //容器组件，处理特定的Host下客户请求，可包含多个Context
        <Context>           //容器组件，为特定的Web应用处理所有的客户请求
        </Context>
      </Host>
    </Engine>
  </Service>
</Server>
```

那么，Tomcat是怎么管理这些容器的呢？你会发现这些容器具有父子关系，形成一个树形结构，你可能马上就想到了设计模式中的组合模式。没错，Tomcat就是用组合模式来管理这些容器的。具体实现方法是，所有容器组件都实现了Container接口，因此组合模式可以使得用户对单容器对象和组合容器对象的使用具有一致性。这里单容器对象指的是最底层的Wrapper，组合容器对象指的是上面的Context、Host或者Engine。Container接口定义如下：

```
public interface Container extends Lifecycle {
    public void setName(String name);
    public Container getParent();
    public void setParent(Container container);
    public void addChild(Container child);
    public void removeChild(Container child);
    public Container findChild(String name);
}
```

正如我们期望的那样，我们在上面的接口看到了getParent、SetParent、addChild和removeChild等方法。你可能还注意到Container接口扩展了LifeCycle接口，LifeCycle接口用来统一管理各组件的生命周期，后面我也用专门的篇幅去详细介绍。

请求定位Servlet的过程

你可能好奇，设计了这么多层次的容器，Tomcat是怎么确定请求是由哪个Wrapper容器里的Servlet来处理的呢？答案是，Tomcat是用Mapper组件来完成这个任务的。

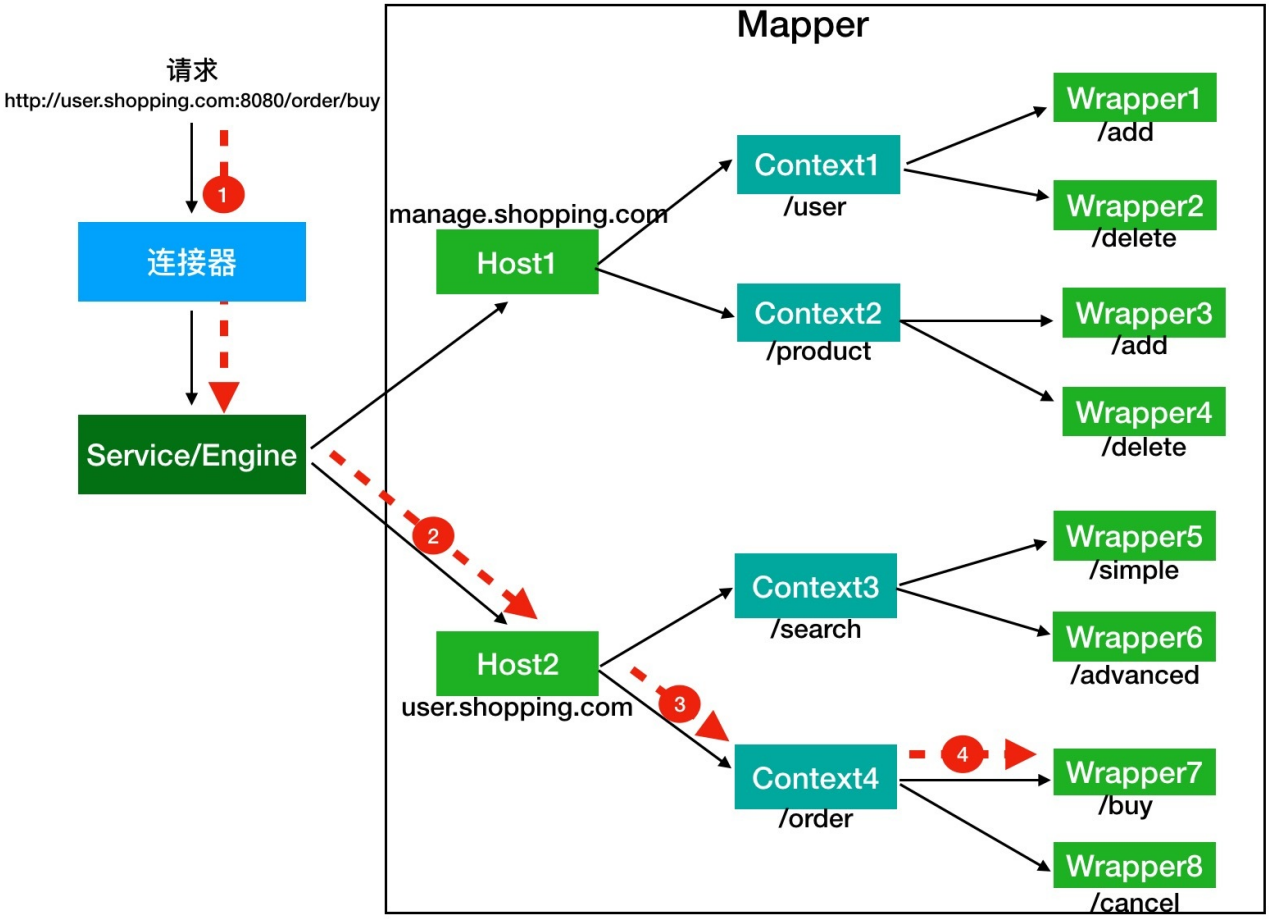
Mapper组件的功能就是将用户请求的URL定位到一个Servlet，它的工作原理是：Mapper组件里保存了Web应用的配置信息，其实就是**容器组件与访问路径的映射关系**，比如Host容器里配置的域名、Context容器里的Web应用路径，以及Wrapper容器里Servlet映射的路径，你可以想象这些配置信息就是一个多层次的Map。

当一个请求到来时，Mapper组件通过解析请求URL里的域名和路径，再到自己保存的Map里去查找，就能定位到一个Servlet。请你注意，一个请求URL最后只会定位到一个Wrapper容器，也就是一个Servlet。

读到这里你可能感到有些抽象，接下来我通过一个例子来解释这个定位的过程。

假如有一个网购系统，有面向网站管理人员的后台管理系统，还有面向终端客户的在线购物系统。这两个系统跑在同一个Tomcat上，为了隔离它们的访问域名，配置了两个虚拟域名：manage.shopping.com和user.shopping.com，网站管理人员通过manage.shopping.com域名访问Tomcat去管理用户和商品，而用户管理和商品管理是两个单独的Web应用。终端客户通过user.shopping.com域名去搜索商品和下订单，搜索功能和订单管理也是两个独立的Web应用。

针对这样的部署，Tomcat会创建一个Service组件和一个Engine容器组件，在Engine容器下创建两个Host子容器，在每个Host容器下创建两个Context子容器。由于一个Web应用通常有多个Servlet，Tomcat还会在每个Context容器里创建多个Wrapper子容器。每个容器都有对应的访问路径，你可以通过下面这张图来帮助你理解。



假如有用户访问一个URL，比如图中的http://user.shopping.com:8080/order/buy，Tomcat如何

仅供个人学习 请勿传播 用户标记 mannix小童鞋

将这个URL定位到一个Servlet呢？

首先，根据协议和端口号选定Service和Engine。

我们知道Tomcat的每个连接器都监听不同的端口，比如Tomcat默认的HTTP连接器监听8080端口、默认的AJP连接器监听8009端口。上面例子中的URL访问的是8080端口，因此这个请求会被HTTP连接器接收，而一个连接器是属于一个Service组件的，这样Service组件就确定了。我们还知道一个Service组件里除了有多个连接器，还有一个容器组件，具体来说就是一个Engine容器，因此Service确定了也就意味着Engine也确定了。

然后，根据域名选定Host。

Service和Engine确定后，Mapper组件通过URL中的域名去查找相应的Host容器，比如例子中的URL访问的域名是`user.shopping.com`，因此Mapper会找到Host2这个容器。

之后，根据URL路径找到Context组件。

Host确定以后，Mapper根据URL的路径来匹配相应的Web应用的路径，比如例子中访问的是/order，因此找到了Context4这个Context容器。

最后，根据URL路径找到Wrapper（Servlet）。

Context确定后，Mapper再根据web.xml中配置的Servlet映射路径来找到具体的Wrapper和Servlet。

看到这里，我想你应该已经了解了什么是容器，以及Tomcat如何通过一层一层的父子容器找到某个Servlet来处理请求。需要注意的是，并不是说只有Servlet才会去处理请求，实际上这个查找路径上的父子容器都会对请求做一些处理。我在上一期说过，连接器中的Adapter会调用容器的Service方法来执行Servlet，最先拿到请求的是Engine容器，Engine容器对请求做一些处理后，会把请求传给自己子容器Host继续处理，依次类推，最后这个请求会传给Wrapper容器，Wrapper会调用最终的Servlet来处理。那么这个调用过程具体是怎么实现的呢？答案是使用Pipeline-Valve管道。

Pipeline-Valve是责任链模式，责任链模式是指在一个请求处理的过程中有很多处理者依次对请求进行处理，每个处理者负责做自己相应的处理，处理完之后将再调用下一个处理者继续处理。

Valve表示一个处理点，比如权限认证和记录日志。如果你还不太理解的话，可以来看看Valve和Pipeline接口中的关键方法。

```
public interface Valve {
  public Valve getNext();
  public void setNext(Valve valve);
  public void invoke(Request request, Response response)
}
```

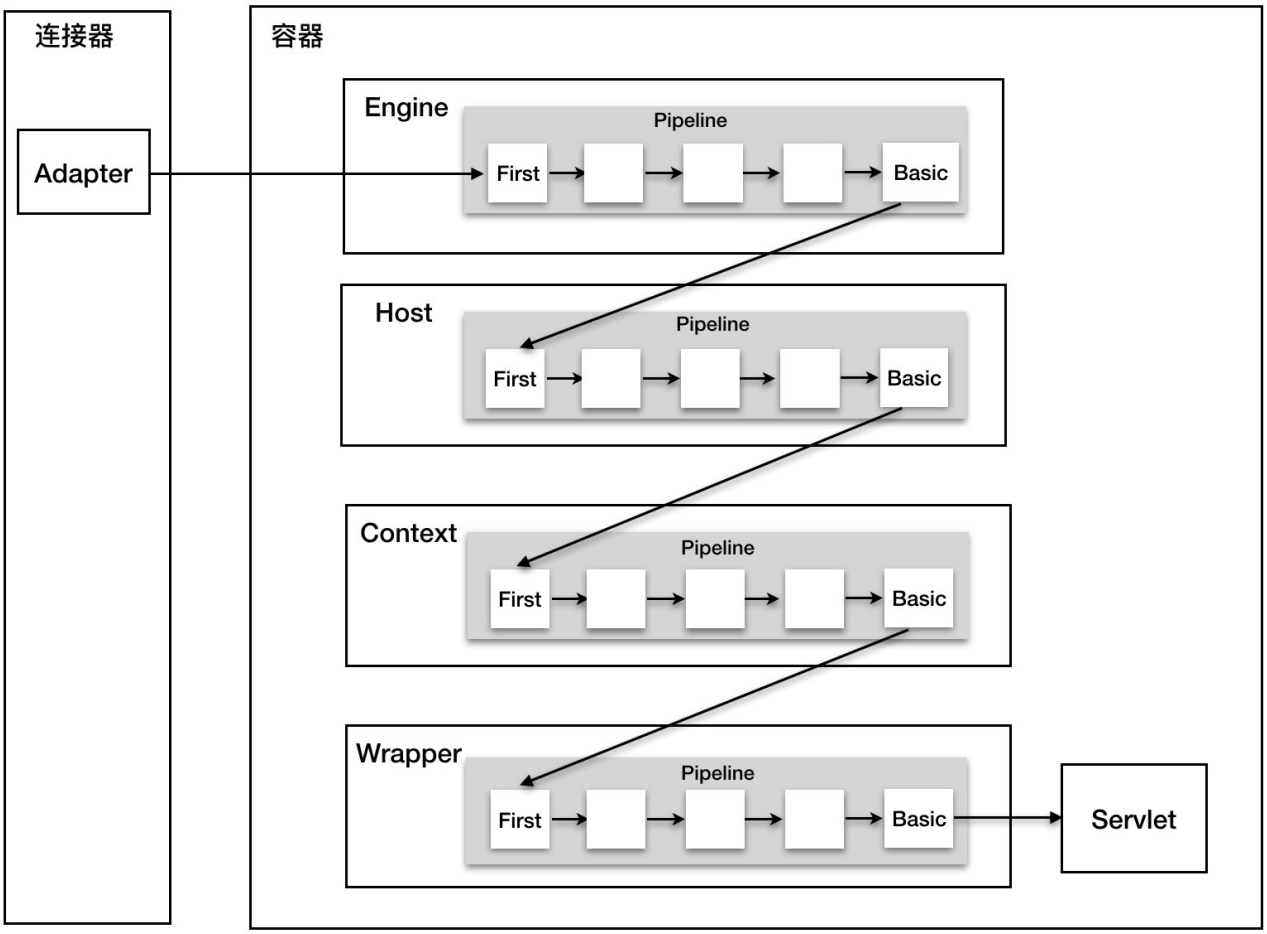
由于Valve是一个处理点，因此invoke方法就是来处理请求的。注意到Valve中有getNext和setNext方法，因此我们大概可以猜到有一个链表将Valve链起来了。请你继续看Pipeline接口：

```
public interface Pipeline extends Contained {
  public void addValve(Valve valve);
  public Valve getBasic();
  public void setBasic(Valve valve);
  public Valve getFirst();
}
```

没错，Pipeline中有addValve方法。Pipeline中维护了Valve链表，Valve可以插入到Pipeline中，对请求做某些处理。我们还发现Pipeline中没有invoke方法，因为整个调用链的触发是Valve来完成的，Valve完成自己的处理后，调用getNext.invoke()来触发下一个Valve调用。

每一个容器都有一个Pipeline对象，只要触发这个Pipeline的第一个Valve，这个容器里Pipeline中的Valve就都会被调用到。但是，不同容器的Pipeline是怎么链式触发的呢，比如Engine中Pipeline需要调用下层容器Host中的Pipeline。

这是因为Pipeline中还有个getBasic方法。这个BasicValve处于Valve链表的末端，它是Pipeline中必不可少的一个Valve，负责调用下层容器的Pipeline里的第一个Valve。我还是通过一张图来解释。



整个调用过程由连接器中的Adapter触发的，它会调用Engine的第一个Valve：

```
// Calling the container
connector.getService().getContainer().getPipeline().getFirst().invoke(request, response);
```

仅供个人学习 请勿传播 用户标记 mannix小童鞋

Wrapper容器的最后一个Valve会创建一个Filter链，并调用doFilter()方法，最终会调到Servlet的service方法。

你可能会问，前面我们不是讲到了Filter，似乎也有相似的功能，那Valve和Filter有什么区别吗？它们的区别是：

- Valve是Tomcat的私有机制，与Tomcat的基础架构/API是紧耦合的。Servlet API是公有的标准，所有的Web容器包括Jetty都支持Filter机制。
- 另一个重要的区别是Valve工作在Web容器级别，拦截所有应用的请求；而Servlet Filter工作在应用级别，只能拦截某个Web应用的所有请求。如果想做整个Web容器的拦截器，必须通过Valve来实现。

本期精华

今天我们学习了Tomcat容器的层次结构、根据请求定位Servlet的过程，以及请求在容器中的调用过程。Tomcat设计了多层容器是为了灵活性的考虑，灵活性具体体现在一个Tomcat实例（Server）可以有多个Service，每个Service通过多个连接器监听不同的端口，而一个Service又可以支持多个虚拟主机。一个URL网址可以用不同的主机名、不同的端口和不同的路径来访问特定的Servlet实例。

请求的链式调用是基于Pipeline-Valve责任链来完成的，这样的设计使得系统具有良好的可扩展性，如果需要扩展容器本身的功能，只需要增加相应的Valve即可。

课后思考

Tomcat内的Context组件跟Servlet规范中的ServletContext接口有什么区别？跟Spring中的ApplicationContext又有什么关系？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- 一路远行 2019-05-23 10:54:34

  1）Servlet规范中ServletContext表示web应用的上下文环境，而web应用对应tomcat的概念是Context，所以从设计上，ServletContext自然会成为tomcat的Context具体实现的一个成员变量。

  2）tomcat内部实现也是这样完成的，ServletContext对应tomcat实现是org.apache.catalina.core.ApplicationContext，Context容器对应tomcat实现是org.apache.catalina.core.StandardContext。ApplicationContext是StandardContext的一个成员变量。

  3）Spring的ApplicationContext之前已经介绍过，tomcat启动过程中ContextLoaderListener会监听到容器初始化事件，它的contextInitialized方法中，Spring会初始化全局的Spring根容器ApplicationContext，初始化完毕后，Spring将其存储到ServletContext中。

  总而言之，Servlet规范中ServletContext是tomcat的Context实现的一个成员变量，而Spring的ApplicationContext是Servlet规范中ServletContext的一个属性。 [16赞]

  作者回复2019-05-23 19:23:56

  👍

- 阿旺 2019-05-23 08:34:54

  你好 请问到业务的controller是从哪部分进去的呢 谢谢 [3赞]

  作者回复2019-05-23 09:27:21

  Wrapper -> Filter -> DispatcherServlet -> Controller

- Joker 2019-05-23 08:20:58

  从具体的数据上来说，Tomcat的Context里面是整个Web应用的参数环境。
  而Servlet规范中的ServletContext接口则既有 Servlet 容器中运行时的参数环境。
  从层级上来说，一个是对应Web应用的，一个则指对应单个Servlet的；一个web应用中可以有多个Servlet。
  ServletContext负责的是servlet运行环境上下信息，不管session管理，不管servlet的加载，他只负责servlet的管理。
  而Tomcat Context需要负责管理session，Servlet的加载和卸载，负责请求信息的处理这些功能。 [2赞]

- 一路远行 2019-05-23 18:59:24

  这段话的描述有些不准确：
  “首先，根据协议和端口号选定 Service 和 Engine。”

  我的理解是，connector配置中只要有端口号就可以确定service和engine，协议的配置只是为了解析，对请求的路由决策没有起到作用。 [1赞]

  作者回复2019-05-23 19:42:46

  你理解对了，这段话的下面就有解释

- 夏天 2019-05-23 08:59:45

  一个tomcat实例下能部署多个工程上去，每个工程下有多个servlet路径，分别也就是context和warpper组件控制，多个host组件目前还没遇到过，能出现多个域名情况 [1赞]

- Joker 2019-05-23 08:38:16

  而Spring中的ApplicationContext则负责管理是Spring容器中的bean，和Tomcat的Context的数据有着本质的区别。 [1赞]

仅供个人学习 请勿传播 用户标记 mannix小童鞋

- 非洲铜 2019-05-23 23:44:29
  public interface Pipeline extends Contained {
  public void addValve(Valve valve);
  public Valve getBasic();
  public void setBasic(Valve valve);
  public Valve getFirst();
  }
  老师，我看了tomcat7的源代码，Pipeline并没有继承contained接口，是版本差异吗？

- allean 2019-05-23 23:38:29
  原文：“所有容器都实现了Container接口，因此组合模式可以使得用户对单容器和组合容器的对象使用具有一致性”
  问题：这段话不太理解，这里说的一致性是指什么，麻烦老师指明😁

- kaiux 2019-05-23 15:26:42
  一直在用Tomcat，现在终于有机会能剥开它仔细看一看了，讲的很棒，醍醐灌顶👍

- Cy190622 2019-05-23 14:29:13
  Tomcat中的Context组件是Web的应用程序，也就是Servlet(在具体代码中是Wrapper)运行的容器，而ServletContext是定义了Servlet的规范，在spring中ApplicationContext 实现了ServletContext。我认为是Context组件为ApplicationContext提供了上下文环境。
  我所理解的url路由过程：
  1.首先根据URL在Mapper组件中找到Servlet的映射关系，应为在初始化Serlvet过程中，Mapper组件中已经有整个容器的映射关系，采用监听的方式，控制各个容器的变化。
  2.然后根据Adapter触发责任链模式来访问Engine、Host、Contextt、Wrapper。
  3.再然后访问Filter链，进行对访问的部分定制化，并调用doFile()。
  4.最后调用Servlet的service方法。
  以上理解不知道是否得当，请老师指教。
  我现在的问题是：
  1.Tomcat中server.xml中顶级组件server和service有什么关系？
  2.后期课程会交代转门交代listener组件和Filter组件吗？

  作者回复2019-05-23 19:16:34
  Spring没有实现ServletContext接口，是Tomcat实现了这个接口，Spring可以拿到通过Request对象拿到ServletContext。

  你可以在server.xml中配置server和service组件。

  后面会谈到Tomcat是如何实现Listener和Filter的

- 永光 2019-05-23 10:15:21
  你好，一个tomcat server对应多个service，这是对应什么样的真实场景呀？

  作者回复2019-05-23 19:28:49
  比如你想在一个Tomcat中运行两个Web应用，又希望两个Web应用使用不同的监听端口，比如一个主程序和一个监控应用..

- -W.LI- 2019-05-23 10:04:10

老师好!Tomcat，server.xml还是搞不太清楚。最外层是server，一个server对应一个Tomcat实例，server下面可以由很多service组件，service组件是区分协议和端口号(问题1:域名访问的话请求到达这里是已经被解析成port了吗？如果已经解析了后面为啥还能拿到二级域名)。每个service对应多个连接器和一个engine。(问题2:为啥要设置多个连接器对应一个engine，是为了提高连接器接受请求的并发吗？)。engine下面对应host，host区分域名(问题3:不同的二级域名对应多个host?)，host下面就是context对应应用通过服务名(webapps下文件夹名区分)（问题4：域名相同路径不同指向不同的服务，是在这配置的么?之前运维同事搞过，原理不太理解），wrapper就是应用程序web.xml中配置的servlet。默认只有一个dispatchservlet。支持对url进行ant风格配置多个。

作者回复2019-05-23 19:39:40
1，DNS解析是在客户端完成的，你需要在客户端把两个域名指向同一个IP，可以通过hosts配置，因此只要你使用的端口相同，两个域名其实访问的是同一个Service组件。而Tomcat设计通过请求URL中Host来转发到不同Host组件的。

2. 不同的连接器代表一种通信的路径，比如同时支持HTTP和HTTPS，不是为了并发。

3. 对

4.域名相同指向不同的Web应用，默认情况下你只需要把你的Web应用全部扔到webapps目录下，Tomcat自动检测，创建相应的Context组件，各个web应用的访问路径就是web应用的目录名。

- Lrwin 2019-05-23 09:43:50
感觉tomcat的多个service没什么用呀？我
Hosts组建和context已经帮我们做好隔离和扩展了、。老师 帮忙回复一下 为什么还要service呢？

作者回复2019-05-23 19:40:17
service主要用不同的端口来访问不同的应用吧

- 逆流的鱼 2019-05-23 09:38:43
好神奇，第一次听说一个Tomcat可以配置或处理多个域名路径的请求？！！？总感觉逻辑怪怪的，即使能现实中也没有这么做的吧？！

作者回复2019-05-23 19:40:52
看来这个问题很热门，已在上面回复

- 天天向上 2019-05-23 08:52:21
Connector和Engine是平级的，并且 Connector可以有多个 容器结构图 xml结构表示的有问题吧

作者回复2019-05-23 09:23:09
是的，谢谢指出，已经在紧急修复了，正确的关系是这样的：
<Server>
<Service>
<Connector>
</Connector>
<Engine>
<Host>
<Context>
</Context>
</Host>
</Engine>
</Service>

</Server>

- Zed 2019-05-23 08:49:24
  宏观上都可以描述为servlet容器，
  区别在于tomcat组件容器是基于web容器，是tomcat组件组合依赖的环境，粒度在组件
  servlet容器是具体应用层面各种servlet的依赖环境，粒度在servlet
  而Spring的ApplicationContext则是Spring实现依赖注入和控制反转的前提条件，是Spring bean的工厂继承于BeanFactory，粒度在bean。

- 磊 2019-05-23 08:34:26
  等了2天，迫不及待的要看下一篇，终于来了！

- 新世界 2019-05-23 08:33:18
  tomcat的context是包含servletContext的，applicationContext只是servletCcontext的一个属性，有一个问题请教：文章中的两个应用为什么四个上下文，一直理解一个应用就一个上下文

  作者回复2019-05-23 08:52:25
  是个四个Web 应用呢

- 东郭 2019-05-23 08:11:23
  后面能否介绍一下tomcat的目录结构呢

  作者回复2019-05-23 08:53:35
  纯手工打造一个Servlet那篇有的

- 轩恒 2019-05-23 07:49:42
  像valve和filter一样，context是容器级的，servletContext是应用级的，而spring的applitionContext应该是对servletContext的封装或实现。